MODUL 1

KONEKTIVITAS BLUETOOTH

TUJUAN

1) Mengetahui teknologi Bluetooth.
2) Memahami proses komunikasi antarperangkat menggunakan Bluetooth.
3) Mengimplementasikan Bluetooth untuk sistem IoT.

REQUIREMENT

1) Arduino IDE/PlatformIO,
2) ESP32,
3) Sensor (opsional),
4) Aktuator (Opsional).

Konektivitas Bluetooth dapat digunakan untuk komunikasi langsung antardua perangkat. Peer-to-peer (P2P) adalah pola komunikasi ketika perangkat berinteraksi secara langsung tanpa server aplikasi pusat. Namun, istilah peran pada Bluetooth bergantung pada jenis teknologinya: Bluetooth Classic dapat menggunakan peran initiator dan acceptor pada koneksi tertentu, sedangkan Bluetooth Low Energy menggunakan peran Central dan Peripheral pada Generic Access Profile (GAP).

1.1 Bluetooth

Bluetooth adalah sebuah media komunikasi yang dapat digunakan untuk menghubungkan sebuah perangkat komunikasi dengan perangkat komunikasi lainnya tanpa menggunakan kabel. Bluetooth biasa digunakan pada smartphone, laptop, komputer dan lain-lain.

Gambar 1.1: Logo Bluetooth

Teknologi Bluetooth dikembangkan pada dekade 1990-an oleh tim insinyur Ericsson, termasuk Jaap Haartsen. Nama Bluetooth diambil dari julukan Raja Harald “Blåtand” Gormsson, yang dikaitkan dengan penyatuan wilayah Denmark dan Norwegia. Nama tersebut digunakan sebagai simbol penyatuan berbagai perangkat melalui satu teknologi komunikasi nirkabel.



Gambar 1.2: Jacobus Cornelis Haartsen

Bluetooth digunakan untuk pertukaran data jarak dekat dan menghubungkan perangkat seperti headset, speaker, keyboard, sensor, serta komputer. Beberapa profil Bluetooth juga mendukung fungsi jaringan, termasuk tethering.

Bluetooth beroperasi pada pita ISM 2,4 GHz. Bluetooth Classic menggunakan frequency hopping, sedangkan Bluetooth Low Energy menggunakan kanal dan mekanisme akses radio yang berbeda. Jangkauan, kecepatan, dan konsumsi daya dipengaruhi oleh versi, mode PHY, daya pancar, antena, kondisi lingkungan, serta implementasi perangkat.

1.2 Jenis-Jenis Bluetooth

1.2.1 Bluetooth 1.0

Pada tahun 1999, versi pertama dari Bluetooth ini mengalami banyak masalah dan produsen mengalami kesulitan untuk menciptakan sebuah produk yang bisa saling berhubungan antara satu sama lain dengan benar. Versi 1.0 dan versi perbaikannya 1.0B bisa dibilang mengalami kegagalan.

Lalu pada tahun 2001 Bluetooth 1.1 pun diciptakan dengan perbaikan dari versi pendahulunya. Selain itu, Bluetooth 1.1 juga telah memiliki standar yang dinamakan IEEE 802.15.1-2002.

Pada tahun 2003, Bluetooth 1.2 diperkenalkan dengan peningkatan proses discovery, Adaptive Frequency Hopping (AFH), serta perbaikan kualitas koneksi. Perangkat Bluetooth dirancang untuk mempertahankan interoperabilitas dengan fitur versi sebelumnya, meskipun keberhasilan koneksi tetap bergantung pada profil dan fitur yang didukung oleh kedua perangkat.

1.2.2 Bluetooth 2.0

Pada tahun 2004, Bluetooth 2.0 diperkenalkan dengan Enhanced Data Rate (EDR) sebagai fitur opsional untuk meningkatkan laju data dan efisiensi waktu transmisi. Penggunaan EDR tidak menghilangkan dukungan terhadap Basic Rate, sehingga perangkat dapat tetap berkomunikasi menggunakan fitur yang sama-sama didukung.

Tiga tahun selanjutnya, Bluetooth mengeluarkan versi terbarunya yaitu Bluetooth 2.1. Bluetooth memiliki teknologi baru, yaitu Secure Simple Pairing (SSP) untuk membuat pairing lebih cepat dan aman. Bluetooth juga memiliki teknologi baru yang lainnya yaitu Extended Inquiry Response (EIR), yang membantu pengguna untuk mengetahui informasi perangkat yang akan dihubungkan sebelum terkoneksi dan konsumsi daya lebih hemat.

1.2.3 Bluetooth 3.0

Pada tahun 2009, Bluetooth 3.0 + High Speed (HS) memperkenalkan Alternate MAC/PHY (AMP). Kanal Bluetooth digunakan untuk discovery, koneksi, dan pengelolaan, sedangkan pemindahan data berkecepatan tinggi dapat dialihkan ke radio IEEE 802.11 pada perangkat yang mendukung fitur HS. Laju teoretis yang dipromosikan mencapai sekitar 24 Mbps, tetapi fitur ini bersifat opsional.

1.2.4 Bluetooth 4.0 (Bluetooth Low Energy)

Pada tahun 2010, Bluetooth Core Specification 4.0 memperkenalkan Bluetooth Low Energy (BLE) sebagai sistem radio berdaya rendah di samping Bluetooth Classic. BLE dirancang agar perangkat dapat beroperasi dengan konsumsi energi rendah, termasuk perangkat yang menggunakan baterai kecil. BLE 4.x menggunakan LE 1M PHY dengan laju simbol 1 Msym/s; throughput aplikasi dan jangkauan aktual bergantung pada konfigurasi serta kondisi lingkungan.

Bluetooth 4.1, yang diadopsi pada tahun 2013, meningkatkan koeksistensi dengan jaringan seluler, fleksibilitas peran perangkat, dan pengelolaan koneksi. Perbaikan tersebut tidak berarti koneksi Bluetooth selalu hidup atau mati secara otomatis; perilaku daya tetap ditentukan oleh implementasi perangkat dan aplikasinya.

Bluetooth 4.2, yang diadopsi pada tahun 2014, menambahkan peningkatan pada kapasitas paket data BLE, privasi, keamanan koneksi, dan dukungan Internet Protocol Support Profile. Peningkatan throughput bergantung pada dukungan fitur Data Length Extension oleh kedua perangkat dan tidak dapat disamakan dengan kecepatan unduh internet.

1.2.5 Bluetooth 5.0

Bluetooth Core Specification 5.0 diadopsi pada tahun 2016 dan memperkenalkan peningkatan penting untuk Bluetooth Low Energy, termasuk LE 2M PHY, LE Coded PHY, dan Extended Advertising.

Fitur-fitur tersebut mendukung beragam kebutuhan IoT, namun keamanan dan ketahanan gangguan tetap bergantung pada konfigurasi serta implementasi perangkat.

Tiga fitur utama yang diperkenalkan adalah:

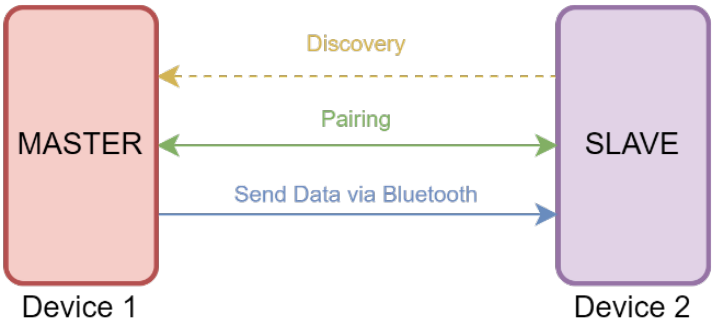
- Hingga 2× laju data melalui LE 2M PHY
- Hingga 4× jangkauan melalui LE Coded PHY pada kondisi tertentu
- Hingga 8× kapasitas data advertising dibandingkan advertising legacy

Bluetooth 5.0 tetap mendukung LE 1M PHY dan menambahkan LE 2M PHY untuk laju data lebih tinggi. LE Coded PHY dapat memperluas jangkauan dengan menambahkan redundansi, tetapi menurunkan laju data efektif. Perangkat hanya dapat menggunakan fitur tersebut apabila kedua sisi mendukung dan mengaktifkannya.

1.2.6 Bluetooth 6.0

Versi terbaru dari Bluetooth dirilis pada tahun 2024, berfokus pada ketelitian pada pencarian perangkat, latensi yang lebih rendah, dan peningkatan efisiensi pada pencarian perangkat. Fitur baru dari versi ini meliputi Channel Sounding, dimana bluetooth menggunakan pencarian berdasarkan fasa serta pengukuran waktu berdasarkan waktu tiba dan datang. Hal ini memungkinkan pencarian perangkat dengan presisi centimeter. Walaupun Bluetooth 6.0 tidak memberikan penambahan performa data transfer, tetapi versi ini memberikan effisensi daya dan security yang lebih baik dibanding dengan versi terdahulunya.

1.3 Alur Komunikasi Bluetooth



Gambar 1.3: Alur Komunikasi Bluetooth

Langkah-langkah Bluetooth bekerja seperti yang ditunjukkan pada gambar di atas, penjelasannya adalah sebagai berikut:

a. **Discovery**

Perangkat melakukan proses discovery untuk menemukan perangkat lain yang berada dalam jangkauan. Pada Bluetooth Classic, proses ini dapat menggunakan prosedur inquiry, sedangkan Bluetooth Low Energy menggunakan scanning terhadap paket advertising.

b. **Pairing**

Pairing dapat dilakukan setelah perangkat ditemukan apabila aplikasi memerlukan pembentukan kunci dan hubungan tepercaya. Tidak semua koneksi Bluetooth mewajibkan pairing; kebutuhan keamanan ditentukan oleh profil, layanan, dan konfigurasi perangkat.

c. **Connection**

Perangkat kemudian dapat membangun koneksi untuk bertukar data. Peran koneksi tidak menunjukkan arah pengiriman data: kedua perangkat dapat mengirim dan menerima. Pada Bluetooth

Low Energy digunakan istilah Central dan Peripheral, sedangkan istilah peran pada Bluetooth Classic bergantung pada prosedur dan profil yang digunakan.

1.4 Protokol Bluetooth

Bluetooth Special Interest Group (SIG) telah mengembangkan spesifikasi Bluetooth yang berisi tentang protokol yang akan digunakan dalam teknologi Bluetooth ini. Protokol dasar Bluetooth adalah Bluetooth Radio, Baseband dan Link Manager Protocol (LMP) yang disebut protokol inti. Sedangkan protokol yang ada di atasnya adalah protokol-protokol terapan yang dapat diadaptasikan pada arsitektur protokol Bluetooth dan telah dikembangkan oleh organisasi lain seperti ETSI. Radio, baseband dan LMP ekivalen dengan lapis fisik dan data link pada lapis protokol OSI.

Stack protokol Bluetooth dapat dibagi ke dalam empat layer sesuai dengan tujuannya. Berikut protokol-protokol dalam layer-layer di dalam stack protokol Bluetooth yang tertera pada tabel di bawah ini:

Tabel 1.1: Layer Protokol dan stack protokol Bluetooth

Layer Protokol	**Protokol di Stack**
Bluetooth Core Protocols	Baseband, LMP, L2CAP, SDP
Cable Replacement Protocols	RFCOMM
Telephony Control Protocols	TCS Binary, AT-commands
Adopted Protocols	PPP, UDP/TCP/IP, OBEX, WAP, vCard, vCal, IrMC, WAE

Selain itu juga terdapat Host Controller Interface (HCI) yang melakukan perintah-perintah yang menghubungkan control baseband, pengaturan hubungan yang dibangun, dan akses ke hardware dan pengaturan register.

a. **Baseband**

Lapis yang memungkinkan hubungan RF terjadi antara beberapa unit Bluetooth membentuk piconet. Sistem RF dari Bluetooth ini menggunakan frekuensi-hopping-spread spectrum yang mengirimkan data dalam bentuk paket pada time slot dan frekuensi yang telah ditentukan, lapis ini melakukan prosedur pemeriksaan dan paging untuk sinkronisasi transmisi frekuensi hopping dan clock dari perangkat Bluetooth yang berbeda.

b. **Link Manager Protocol (LMP)**

Link Manager Protocol mengatur pembentukan dan pengelolaan tautan pada Bluetooth Classic. Fungsinya mencakup autentikasi, pengaturan enkripsi, negosiasi parameter tautan, dan pengendalian fitur baseband.

c. **Logical Link Control and Adaptation Protocol (L2CAP)**

Paket L2CAP membawa muatan yang penting yang dibawa ke layer protokol yang lebih tinggi.

d. **Service Discovery Protocol (SDP)**

Protokol ini digunakan untuk menemukan layanan yang disediakan oleh perangkat Bluetooth Classic beserta atribut yang diperlukan untuk mengakses layanan tersebut.

e. **Cable Replacement Protocol (RFCOMM)**

RFCOMM adalah emulasi jalur serial.

f. **Telephony Control Protocol**

Telephony Control Specification Binary (TCS BIN) dan perintah AT digunakan pada profil telepon tertentu untuk mengendalikan panggilan serta fungsi modem.

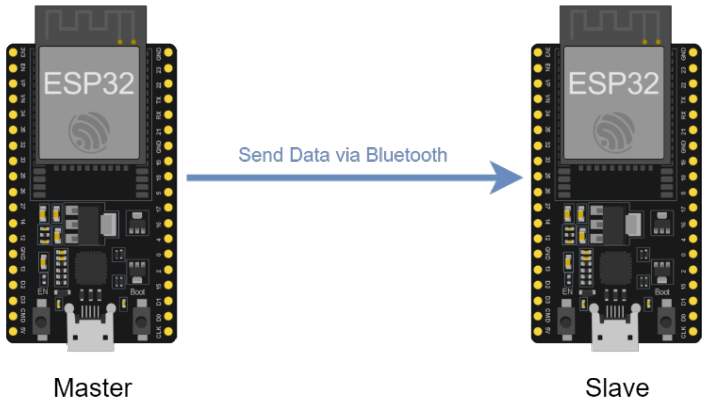
g. **Adopted Protocols**

Bluetooth juga mendukung protokol PPP, TCP/UDP/IP, OBEX dan WAP untuk memaksimalkan interoperabilitasnya.

h. **Radio Frequency (RF)**

Radio Frequency merupakan lapisan terendah dari spesifikasi Bluetooth dan menyediakan komunikasi nirkabel pada pita Industrial, Scientific, and Medical (ISM) 2,4 GHz. Bluetooth Classic menggunakan 79 kanal RF dengan lebar 1 MHz dan mekanisme frequency hopping, sedangkan Bluetooth Low Energy menggunakan 40 kanal RF dengan lebar 2 MHz.

1.5 Implementasi peer-to-peer Bluetooth



Gambar 1.4: Visual P2P

Dalam implementasi peer-to-peer Bluetooth ini, master akan mengirimkan data ke slave menggunakan mikrokontroler ESP32. Jika ingin menggunakan mikrokontroller yang lain, pastikan perangkat mensupport jenis Bluetooth yang ingin dipakai. Bila perangkat hanya mensupport Bluetooth versi 4.0 ke atas tanpa adanya dual-mode, maka perangkat tersebut hanya dapat menggunakan BLE dan begitu sebaliknya. Bila perangkat memiliki dual-mode support (Bluetooth Classic dan BLE), maka perangkat tersebut dapat menggunakan implementasi Bluetooth Classic dan BLE.

1.5.1 Menggunakan Bluetooth Serial

Konfigurasi Bluetooth Serial tergolong sederhana dibandingkan dengan BLE karena menggunakan konsep **Master** dan **Slave** seperti protokol serial komunikasi lainnya. Berikut adalah contoh konfigurasi master dan slave pada ESP32.

Informasi
Library Bluetooth Serial yang akan dipakai adalah library builtin dari ESP32:
https://github.com/espressif/arduino-esp32/tree/master/libraries/BluetoothSerial

- **Master**

Master merupakan bagian yang akan mencari koneksi kepada Slave dan akan mengirimkan data kepada mereka.

Inisialisasi Source code untuk Master/Host

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String masterName = "ESP32-Master"; // Nama Master ESP
String slaveName = "ESP32-Slave";
bool connected = false;
int value = 0;
```

Dalam menginisialisasi P2P ESP32, kita menggunakan library `BluetoothSerial.h` agar bisa mengakses kode-kode yang mendukung fungsi Bluetooth. Setelah itu, siapkan variabel-variabel seperti variabel untuk mengakses fungsi Bluetooth seperti variabel `SerialBT` yang dapat mengakses BluetoothSerial, variabel `String masterName` yang digunakan untuk mencari nama perangkat yang dituju yaitu `ESP32-Slave`, `bool connected` untuk konektivitas dan `int value` untuk digunakan sebagai penyimpan nilai yang akan dikirimkan ke Slave.

Setup pada Source code Master

```
void setup() {
  Serial.begin(115200);

  SerialBT.begin(masterName, true); // Memulai Bluetooth
  Serial.println("The device started in master mode, make sure remote BT device is on!");

  connected = SerialBT.connect(slaveName); // Periksa koneksi Bluetooth
  if(connected) {
    Serial.println("Connected Successfully!");
  } else {
    while(!SerialBT.connected(10 * 1000)) { // Menunggu 10 detik
      Serial.println("Failed to connect!");
      Serial.println("Make sure remote device is available and in range, then restart");
    }
  }
  SerialBT.connect();
}
```

Pada `setup()`, kita menggunakan `SerialBT.begin("ESP32Master", true)` untuk menentukan nama yang akan terlihat oleh perangkat lain. Untuk `connected = SerialBT.connect(Name)` digunakan sebagai koneksi kepada Bluetooth yang dituju dan indikator untuk terkoneksi atau tidak. Dan dilanjutkan dengan menghubungkan dengan Bluetooth pada `SerialBT.connect();`

Loop pada Source Code Master

```
void loop() {
  if(value <= 10){
    SerialBT.println(value); // Mengirimkan data melalui Bluetooth
    Serial.println(value);
    value++;

    delay(1000);
  }
  else {
    value = value - 10;
  }
}
```

Pada tahap perulangan, kita akan mencoba mengirimkan kepada `Slave` nilai dari variabel `value` hingga bernilai 10. Kita menggunakan `SerialBT.println(value)` yang di mana digunakan untuk mengirimkan nilai kepada `Slave` berdasarkan nilai dari variabel `value`.

- **Slave**

Slave merupakan bagian yang akan menerima komunikasi dari Master.

Inisialisasi Source code untuk Slave

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String slaveName = "ESP32-Slave"; // Nama Slave ESP
int value = 0;
String slaveValueStr;
int slaveValueInt;
```

Pada proses inisialisasi pada Slave, kita kembali menggunakan library `BluetoothSerial.h`.. Kemudian, inisialisasikan kembali `BluetoothSerial` pada variabel `SerialBT` dan siapkan variabel-variabel yang akan digunakan seperti `int value`, `String slaveValueStr` yang akan digunakan sebagai data yang diterima berupa string, dan `int slaveValueInt` yang akan digunakan untuk menyimpan nilai int yang telah diubah dari string ke int. Terakhir, `String masterName = "ESP32-BT-Slave"` merupakan inisial untuk nama perangkat kita.

Setup pada Source code Slave

```
void setup() {
  Serial.begin(115200);

  SerialBT.begin(slaveName); // Memulai Bluetooth sebagai perangkat penerima
  Serial.println("The device started, now you can pair it with bluetooth!");
}
```

Pada setup, kita akan mencoba menyambungkan koneksi dengan `SerialBT.begin(slaveName)` dan menentukan nama yang akan terlihat oleh perangkat lain berdasarkan nilai dari variabel `slaveName`.

Loop pada Source code Slave

```
void loop() {
  if(SerialBT.available()){
    slaveValueStr = SerialBT.readStringUntil('\n'); // Membaca data dari Master
    slaveValueInt = slaveValueStr.toInt();

    Serial.print("Data dari Master : ");
    Serial.println(slaveValueInt);
  }
}
```

Pada perulangan, kita mencoba untuk memeriksa apakah Bluetooth sudah terhubung dengan perangkat master atau tidak dengan `if(SerialBT.available())`. Kemudian, variabel `slaveValueStr` akan membaca data string hingga akhir kalimat sebelum baris baru. Dari Master, kita menggunakan `SerialBT.readStringUntil('\n')`, lalu kita akan mengubah nilai `String` yang dibaca dari master menjadi `int` yang akan disimpan ke variabel `slaveValueInt` menggunakan `slaveValueStr.toInt()` sehingga akan menampilkan hasil Serial Monitor berikut.

```
Data Dari Master : 1
Data Dari Master : 2
Data Dari Master : 3
Data Dari Master : 4
Data Dari Master : 5
Data Dari Master : 6
Data Dari Master : 7
Data Dari Master : 8
Data Dari Master : 9
Data Dari Master : 10
Data Dari Master : 1
```

Gambar 1.5: Output pada Serial Monitor

Gambar di atas merupakan tampilan Serial Monitor pada Slave jika menerima data yang dikirimkan dari Master berhasil.

1.5.2 Menggunakan Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) menggunakan model **Central** dan **Peripheral**. Perangkat **Central/Client** melakukan pemindaian dan menghubungkan diri ke perangkat BLE, sedangkan **Peripheral/Server** melakukan **advertising** serta menyediakan data melalui struktur GATT (**Generic Attribute Profile**).

Pada contoh ini, **Peripheral/Server** menghasilkan nilai acak dan mengirimkannya melalui notifikasi. **Central/Client** mencari perangkat dengan **Service UUID** yang sesuai, melakukan koneksi, lalu menerima data melalui fungsi callback.

Informasi
Untuk mendapatkan nilai UUID random, website di bawah dapat digunakan:
https://uuidgenerator.net

- **Central/Client**

Central bertugas memindai perangkat BLE, menemukan Peripheral berdasarkan **Service UUID**, melakukan koneksi, dan menerima notifikasi dari characteristic.

Library, Variabel, dan Deklarasi BLE Central/Client

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEClient.h>
#include <BLEAdvertisedDevice.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

static BLEAddress *pServerAddress = nullptr;
static BLERemoteCharacteristic *pRemoteCharacteristic = nullptr;
static BLEClient *pClient = nullptr;

static bool doConnect = false;
static bool connected = false;
```

Library tersebut menyediakan fungsi scanning, koneksi client, dan akses ke characteristic pada BLE. Variabel pointer menyimpan alamat perangkat, objek client, dan characteristic remote. Sementara itu, `doConnect` dan `connected` mengatur proses serta status koneksi.

Fungsi Notifikasi BLE Central/Client

```
static void notifyCallback(
  BLERemoteCharacteristic* pBLERemoteCharacteristic,
  uint8_t* pData,
  size_t length,
  bool isNotify) {

  String receivedData = "";
  for (size_t i = 0; i < length; i++) {
    receivedData += (char)pData[i];
  }

  Serial.print("Received from Slave: ");
  Serial.println(receivedData);
}
```

Fungsi `notifyCallback()` dijalankan secara otomatis ketika Central menerima notifikasi dari characteristic milik Peripheral. Parameter `pData` berisi data yang diterima dalam bentuk array byte, sedangkan `length` menunjukkan jumlah byte yang diterima. Setiap byte diubah menjadi karakter menggunakan `static_cast<char>()`, kemudian digabungkan ke dalam variabel `receivedData`.

Setelah seluruh data selesai diproses, nilainya ditampilkan melalui Serial Monitor.

Callback Koneksi BLE Central/Client

```
class MyClientCallbacks : public BLEClientCallbacks {
  void onConnect(BLEClient *pClient) override {
    connected = true;
    Serial.println("Connected to Peripheral!");
  }

  void onDisconnect(BLEClient *pClient) override {
    connected = false;
    Serial.println("Disconnected from Peripheral!");
  }
};
```

Kelas `MyClientCallbacks` digunakan untuk mendeteksi perubahan status koneksi antara Central dan Peripheral. Fungsi `onConnect()` dijalankan ketika Central berhasil terhubung ke Peripheral. Pada kondisi ini, variabel `connected` diubah menjadi `true`. Fungsi `onDisconnect()` dijalankan ketika koneksi terputus. Variabel `connected` diubah menjadi `false` agar Central dapat menjalankan proses scanning ulang.

Callback Scanning BLE Central/Client

```
class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
 void onResult(BLEAdvertisedDevice advertisedDevice) override {
    Serial.print("Found device: ");
    Serial.println(advertisedDevice.toString().c_str());

    // Cek apakah device memiliki service UUID yang sesuai
    if (
      advertisedDevice.haveServiceUUID() &&
      advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))
    ) {
      Serial.println("Target Peripheral found!");
      BLEDevice::getScan()->stop();

      if (pServerAddress != nullptr) {
        delete pServerAddress;
      }

```

```
        pServerAddress = new BLEAddress(advertisedDevice.getAddress());

        doConnect = true;
      }
    }
};
```

Kelas `MyAdvertisedDeviceCallbacks` digunakan untuk memproses setiap perangkat BLE yang ditemukan selama scanning. Fungsi `onResult()` dijalankan setiap kali scanner menemukan perangkat BLE. Informasi perangkat tersebut ditampilkan melalui `advertisedDevice.toString()`. Fungsi `haveServiceUUID()` memeriksa apakah perangkat mengiklankan Service UUID. Setelah itu, `isAdvertisingService()` membandingkan UUID perangkat dengan `SERVICE_UUID` yang dicari.

Jika UUID sesuai, scanning dihentikan menggunakan `BLEDevice::getScan()->stop()`. Alamat Peripheral disimpan dalam `pServerAddress`, kemudian `doConnect` diubah menjadi `true` agar proses koneksi dijalankan pada bagian `loop()`.

Inisialisasi dan Koneksi BLE Central/Client

```
bool connectToServer(BLEAddress address) {
  Serial.print("Connecting to Peripheral: ");
  Serial.println(address.toString().c_str());

  pClient = BLEDevice::createClient();
  pClient->setClientCallbacks(new MyClientCallbacks());

  if (!pClient->connect(address)) {
    Serial.println("Connection failed!");
    return false;
  }

  BLERemoteService *pRemoteService = pClient->getService(BLEUUID(SERVICE_UUID));
 if (pRemoteService == nullptr) {
    Serial.println("Service not found!");
    pClient->disconnect();
    return false;
  }

  pRemoteCharacteristic = pRemoteService->getCharacteristic(BLEUUID(CHARACTERISTIC_UUID));
 if (pRemoteCharacteristic == nullptr) {
    Serial.println("Characteristic not found!");
    pClient->disconnect();
    return false;
  }

  if (!pRemoteCharacteristic->canNotify()) {
    Serial.println("Characteristic does not support notifications!");
    pClient->disconnect();
    return false;
  }

  pRemoteCharacteristic->registerForNotify(notifyCallback);
  connected = true;

  Serial.println("Notification enabled!");
  return true;
}
```

`connectToServer()` menghubungkan Central, mencari service dan characteristic, lalu mendaftarkan callback notifikasi. Fungsi ini akan digunakan di fungsi `loop()` nanti.

Setup pada Source Central/Client

```
void setup() {
  Serial.begin(115200);
  Serial.println("=== BLE Central/Client ===");

  // Inisialisasi BLE Master
  BLEDevice::init("ESP32_Central");

  // Mulai Scanning
  BLEScan *pBLEScan = BLEDevice::getScan();
  pBLEScan->setAdvertisedDeviceCallbacks(
    new MyAdvertisedDeviceCallbacks()
  );
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(80);
  pBLEScan->setActiveScan(true);

  Serial.println("Scanning for Peripheral...");
  pBLEScan->start(5, false); // Scan selama 5 detik
}

```

Pada `setup()`, kita menginisialisasi Serial dan BLE dan melakukan registrasi callback yang telah kita buat ke dalam pointer `pBLEScan` melalui `setAdvertisedDeviceCallbacks()`. Selain itu, interval pengiriman dan proses pencarian peripheral juga dilaksanakan.

Loop BLE Central/Client

```
void loop() {
  // Jika menemukan device yang cocok, coba koneksi
  if (doConnect && pServerAddress != nullptr) {
    if (connectToServer(*pServerAddress)) {
      Serial.println("Ready to receive data!");
    } else {
      Serial.println("Connection failed. Scanning again...");
    }

    doConnect = false;
  }

  // Jika terputus, coba scan lagi
  if (!connected && !doConnect) {
    BLEDevice::getScan()->start(5, false);
  }

  delay(1000);
}
```

Pada `loop()`, Central mencoba terhubung setelah target ditemukan. Jika belum terhubung, proses scanning akan dijalankan kembali.

- **Peripheral/Server**

Peripheral melakukan **advertising**, menyediakan service dan characteristic, lalu mengirim nilai acak kepada Central melalui notifikasi.

Library, Variabel, dan Deklarasi BLE Peripheral/Server

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"
```

```

BLEServer *pServer = nullptr;
BLECharacteristic *pCharacteristic = nullptr;

bool deviceConnected = false;
bool oldDeviceConnected = false;

unsigned long previousSendTime = 0;
const unsigned long sendInterval = 1000;
```

Library BLE Server digunakan untuk membuat server, service, characteristic, dan descriptor notifikasi. Variabel koneksi menyimpan status Central, sedangkan variabel waktu mengatur interval pengiriman data.

Callback BLE Peripheral/Server

```
class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer *pServer) override {
    deviceConnected = true;
    Serial.println("Central connected!");
  }

  void onDisconnect(BLEServer *pServer) override {
    deviceConnected = false;
    Serial.println("Central disconnected!");
  }
};
```

Callback server dijalankan saat Central terhubung atau terputus. Nilai `deviceConnected` digunakan oleh `loop()` untuk menentukan apakah notifikasi dapat dikirim.

Inisialisasi BLE Peripheral/Server

```
void BLE_init() {
  BLEDevice::init("ESP32_Peripheral");

  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService =
    pServer->createService(BLEUUID(SERVICE_UUID));

  // Membuat characteristic (data) di dalam service
  // PROPERTY_READ     -> Client bisa baca data
  // PROPERTY_WRITE    -> Client bisa kirim data ke ESP32
  // PROPERTY_NOTIFY   -> ESP32 kirim data otomatis (tanpa ACK)
  // PROPERTY_INDICATE -> ESP32 kirim data otomatis (dengan ACK)
  // PROPERTY_WRITE_NR -> Client kirim data tanpa konfirmasi
  pCharacteristic = pService->createCharacteristic(
    BLEUUID(CHARACTERISTIC_UUID),
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY
  );

  pCharacteristic->addDescriptor(new BLE2902());
  pCharacteristic->setValue("0");

  pService->start();

  BLEAdvertising *pAdvertising =
    BLEDevice::getAdvertising();

  pAdvertising->addServiceUUID(BLEUUID(SERVICE_UUID));
  pAdvertising->setScanResponse(true);

```

```
  BLEDevice::startAdvertising();

  Serial.println("Peripheral ready. Waiting for Central...");
}

void setup() {
  Serial.begin(115200);
  randomSeed(micros());

  BLE_init();
}
```

`BLE_init()` membuat server, service, dan characteristic, menambahkan descriptor notifikasi, lalu memulai advertising. Fungsi `setup()` akan menyiapkan Serial, generator nilai acak untuk percobaan pengiriman data, dan konfigurasi BLE.

Loop BLE Peripheral/Server

```
void loop() {
  if (
    deviceConnected &&
    millis() - previousSendTime >= sendInterval
  ) {
    previousSendTime = millis();

    String value = String(random(0, 101));

    pCharacteristic->setValue(value.c_str());
    pCharacteristic->notify();

    Serial.print("Sent: ");
    Serial.println(value);
  }

  if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();

    Serial.println("Advertising restarted...");
    oldDeviceConnected = false;
  }

  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = true;
  }
}
```

Pada `loop()`, Peripheral mengirim nilai acak setiap satu detik ketika Central terhubung. Jika koneksi terputus, advertising diaktifkan kembali agar perangkat dapat ditemukan dan dihubungkan ulang.

Dengan alur tersebut, Peripheral mengirimkan data dan Central menerimanya melalui notifikasi. Komunikasi tidak menggunakan aliran serial seperti Bluetooth Classic, tetapi menggunakan service, characteristic, dan callback pada GATT.

LAMPIRAN MODUL 1

Source Code Bluetooth Master

Bluetooth Master

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String masterName = "ESP32-Master"; // Nama Master ESP
String slaveName = "ESP32-Slave";
bool connected = false;
int value = 0;

void setup() {
  Serial.begin(115200);

  SerialBT.begin(masterName, true); // Memulai Bluetooth
  Serial.println("The device started in master mode, make sure remote BT device is on!");

  connected = SerialBT.connect(slaveName); // Periksa koneksi Bluetooth
  if(connected) {
    Serial.println("Connected Successfully!");
  } else {
    while(!SerialBT.connected(10 * 1000)) { // Menunggu 10 detik
      Serial.println("Failed to connect!");
      Serial.println("Make sure remote device is available and in range, then restart");
    }
  }
  SerialBT.connect();
}

void loop() {
  if(value <= 10){
    SerialBT.println(value); // Mengirimkan data melalui Bluetooth
    Serial.println(value);
    value++;

    delay(1000);
  }
  else {
    value = value - 10;
  }
}
```

Source Code Bluetooth Slave

Bluetooth Slave

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String slaveName = "ESP32-Slave"; // Nama Slave ESP
int value = 0;
String slaveValueStr;
int slaveValueInt;

void setup() {
  Serial.begin(115200);
  SerialBT.begin(slaveName); // Memulai Bluetooth sebagai perangkat penerima
  Serial.println("The device started, now you can pair it with bluetooth!");
}

void loop() {
  if(SerialBT.available()){
```

```
    slaveValueStr = SerialBT.readStringUntil('\n'); // Membaca data dari Master
    slaveValueInt = slaveValueStr.toInt();

    Serial.print("Data dari Master : ");
    Serial.println(slaveValueInt);
  }
}
```

Source Code BLE Central/Client

BLE Central/Client

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEClient.h>
#include <BLEAdvertisedDevice.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

static BLEAddress *pServerAddress;   // Alamat device BLE Slave
static boolean doConnect = false;    // Flag untuk mulai koneksi
static boolean connected = false;    // Status koneksi
static BLERemoteCharacteristic* pRemoteCharacteristic; // Karakteristik remote
BLEClient*  pClient;                 // Objek client BLE

static void notifyCallback(
  BLERemoteCharacteristic* pBLERemoteCharacteristic,
  uint8_t* pData,
  size_t length,
  bool isNotify) {

  //  Letakkan logika kontrol di sini

  String receivedData = "";
  for (size_t i = 0; i < length; i++) {
    receivedData += (char)pData[i];
  }

  Serial.print("Received from Slave: ");
  Serial.println(receivedData);
}

class MyClientCallbacks : public BLEClientCallbacks {
  void onConnect(BLEClient *pClient) override {
    connected = true;
    Serial.println("Connected to Peripheral!");
  }

  void onDisconnect(BLEClient *pClient) override {
    connected = false;
    Serial.println("Disconnected from Peripheral!");
  }
};

bool connectToServer(BLEAddress pAddress) {
  Serial.print("Connecting to Slave: ");
  Serial.println(pAddress.toString().c_str());

  // Buat client baru
  pClient = BLEDevice::createClient();
  pClient->setClientCallbacks(new MyClientCallbacks());

  // Coba koneksi ke alamat BLE Slave
  if (!pClient->connect(pAddress)) {
    Serial.println("Connection failed!");
```

```
    return false;
  }

  Serial.println("Connected to Slave!");

  // Ambil service dari Slave
  BLERemoteService* pRemoteService = pClient->getService(SERVICE_UUID);
  if (pRemoteService == nullptr) {
    Serial.println("Service not found!");
    pClient->disconnect();
    return false;
  }

  // Ambil characteristic dari service
  pRemoteCharacteristic = pRemoteService->getCharacteristic(CHARACTERISTIC_UUID);
  if (pRemoteCharacteristic == nullptr) {
    Serial.println("Characteristic not found!");
    pClient->disconnect();
    return false;
  }

  // Aktifkan notifikasi
  if (!pRemoteCharacteristic->canNotify()) {
    Serial.println("Characteristic does not support notifications!");
    pClient->disconnect();
    return false;
  }

  pRemoteCharacteristic->registerForNotify(notifyCallback);
  connected = true;

  Serial.println("Notification enabled!");
  return true;
}

class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
 void onResult(BLEAdvertisedDevice advertisedDevice) override {
    Serial.print("Found device: ");
    Serial.println(advertisedDevice.toString().c_str());

    // Cek apakah device memiliki service UUID yang sesuai
    if (
      advertisedDevice.haveServiceUUID() &&
      advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))
    ) {
      Serial.println("Target Peripheral found!");
      BLEDevice::getScan()->stop();

      if (pServerAddress != nullptr) {
        delete pServerAddress;
      }

      pServerAddress = new BLEAddress(advertisedDevice.getAddress());
      doConnect = true;
    }
  }
};

void setup() {
  Serial.begin(115200);
  Serial.println("=== BLE Central/Client ===");

  // Inisialisasi BLE Master
  BLEDevice::init("ESP32_Central");

  // Mulai Scanning
  BLEScan *pBLEScan = BLEDevice::getScan();
```

```
  pBLEScan->setAdvertisedDeviceCallbacks(
    new MyAdvertisedDeviceCallbacks()
  );
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(80);
  pBLEScan->setActiveScan(true);

  Serial.println("Scanning for Peripheral...");
  pBLEScan->start(5, false); // Scan selama 5 detik
}

void loop() {
  // Jika menemukan device yang cocok, coba koneksi
  if (doConnect && pServerAddress != nullptr) {
    if (connectToServer(*pServerAddress)) {
      Serial.println("Ready to receive data!");
    } else {
      Serial.println("Connection failed. Scanning again...");
    }

    doConnect = false;
  }

  // Jika terputus, coba scan lagi
  if (!connected && !doConnect) {
    BLEDevice::getScan()->start(5, false);
  }

  delay(1000);
}
```

Source Code BLE Peripheral/Server

BLE Peripheral/Server

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define SERVICE_UUID "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

BLEServer *pServer = nullptr;
BLECharacteristic *pCharacteristic = nullptr;

bool deviceConnected = false;
bool oldDeviceConnected = false;

unsigned long previousSendTime = 0;
const unsigned long sendInterval = 1000;

class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer *pServer) {
    deviceConnected = true;            // Set status terhubung
    Serial.println("Master connected!");
  }

  void onDisconnect(BLEServer *pServer) {
    deviceConnected = false;           // Set status terputus
    Serial.println("Master disconnected!");
  }
};

void BLE_init() {
  BLEDevice::init("ESP32_Peripheral");

```

```
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService =
    pServer->createService(BLEUUID(SERVICE_UUID));

  // Membuat characteristic (data) di dalam service
  // PROPERTY_READ     -> Client bisa baca data
  // PROPERTY_WRITE    -> Client bisa kirim data ke ESP32
  // PROPERTY_NOTIFY   -> ESP32 kirim data otomatis (tanpa ACK)
  // PROPERTY_INDICATE -> ESP32 kirim data otomatis (dengan ACK)
  // PROPERTY_WRITE_NR -> Client kirim data tanpa konfirmasi
  pCharacteristic = pService->createCharacteristic(
    BLEUUID(CHARACTERISTIC_UUID),
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY
  );

  pCharacteristic->addDescriptor(new BLE2902());
  pCharacteristic->setValue("0");

  pService->start();

  BLEAdvertising *pAdvertising =
    BLEDevice::getAdvertising();

  pAdvertising->addServiceUUID(BLEUUID(SERVICE_UUID));
  pAdvertising->setScanResponse(true);

  BLEDevice::startAdvertising();

  Serial.println("Peripheral ready. Waiting for Central...");
}

void setup() {
  Serial.begin(115200);
  randomSeed(micros());

  BLE_init();
}

void loop() {
  if (
    deviceConnected &&
    millis() - previousSendTime >= sendInterval
  ) {
    previousSendTime = millis();

    String value = String(random(0, 101));

    // BLE hanya bisa mengirim jenis string C (null terminated "\0")
    pCharacteristic->setValue(value.c_str());
    pCharacteristic->notify();

    Serial.print("Sent: ");
    Serial.println(value);
  }

  if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();

    Serial.println("Advertising restarted...");
    oldDeviceConnected = false;
  }

  if (deviceConnected && !oldDeviceConnected) {
```

```
100       oldDeviceConnected = true;
101     }
102   }
```